==========

CAPSTONE PROJECT

NOTEBOOK 47: FINAL AUDIO INFERENCE PIPELINE (PATCHED)

==========

Purpose:

This is the final deployment-style inference notebook for the

capstone project.

It allows the user to:

1. Load a single audio file

2. Run multi-window Stage-1 hybrid inference

3. Run safer Stage-2 rare-tail fallback

4. View candidate-label predictions and rare-tail suggestions

5. Optionally compare with ground truth for FMA tracks

6. Save final report-ready outputs

==========

In [1]:

```
# ============================================================
# 0. INSTALL / CHECK REQUIRED PACKAGES
# ============================================================
# This cell installs packages into the exact Python environment
# currently being used by this notebook.

import sys
import importlib.util
import subprocess

required_packages = {
    "joblib": "joblib",
    "librosa": "librosa",
    "numpy": "numpy",
    "pandas": "pandas",
    "scipy": "scipy",
    "sklearn": "scikit-learn",
    "soundfile": "soundfile",
    "audioread": "audioread",
    "tensorflow": "tensorflow==2.20.0"
}

missing_packages = []

for import_name, pip_name in required_packages.items():
    if importlib.util.find_spec(import_name) is None:
        missing_packages.append(pip_name)

if missing_packages:
    print("Installing missing packages:", missing_packages)
    subprocess.check_call([
        sys.executable,
        "-m",
        "pip",
        "install",
        *missing_packages
    ])
else:
    print("All required packages are already installed.")
```

```
print("Python executable being used:")
print(sys.executable)
```

```
Installing missing packages: ['joblib', 'librosa', 'numpy', 'pandas', 'scipy',
'scikit-learn', 'soundfile', 'audioread', 'tensorflow==2.20.0']
Python executable being used:
e:\SCHOOL\Masters\Capstone_FMA_Project\notebook\.venv\Scripts\python.exe
```

In [2]:

```
# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import os
import json
import math
import tempfile
import subprocess
import warnings
from pathlib import Path

import joblib
import librosa
import numpy as np
import pandas as pd
import tensorflow as tf

from scipy.stats import skew, kurtosis

warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning)

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("Seed set to:", SEED)
print("TensorFlow version:", tf.__version__)
```

```
Seed set to: 42
TensorFlow version: 2.20.0
```

In [3]:

```
# ============================================================
# 2. USER SETTINGS
# ============================================================
# MODE OPTIONS:
# - "existing_fma_track" : use a track_id from your processed FMA tables
# - "external_file"      : use your own audio file

MODE = "external_file"

# Option 1: choose an FMA track_id that exists in multilabel_full_master_table.csv
```

```
TRACK_ID_TO_TEST = 568

# Option 2: path to your own external file
# Change this path to whatever song/audio file you want to test.
EXTERNAL_AUDIO_PATH = r"C:\Users\jdevo\Downloads\Bob Marley - Is This Love
(Official Music Video).mp3"

# How many top candidate labels to show
TOP_N_CANDIDATES = 10

# ============================================================
# MULTI-WINDOW SETTINGS
# ============================================================
# The pipeline listens to multiple sections of the audio instead
# of only the first 15 seconds.

WINDOW_SECONDS = 15
MAX_WINDOWS = 4
WINDOW_SELECTION_MODE = "evenly_spaced"   # "evenly_spaced" or "first_n"

# ============================================================
# FINAL WINDOW AGGREGATION SETTINGS
# ============================================================
# A genre is accepted as a final prediction only if:
# 1. Its average hybrid score is above the Stage-1 threshold
# 2. It appears in at least this many windows

MIN_WINDOW_VOTES = 2

# ============================================================
# SAFER STAGE-2 SETTINGS
# ============================================================
# Stage 2 rare-tail suggestions are weak fallback hints.
# These settings prevent tiny/noisy suggestions from appearing.

REQUIRE_ANCHOR_PREDICTED = True
MIN_STAGE2_SCORE = 0.01
LOW_CONFIDENCE_THRESHOLD = 0.50
SUPPRESS_STAGE2_IF_LOW_CONFIDENCE = False

# ============================================================
# AUDIO SETTINGS
# ============================================================

SR = 22050
N_MELS = 64
N_FFT = 2048
HOP_LENGTH = 1024
MAX_FRAMES = int(np.ceil((WINDOW_SECONDS * SR) / HOP_LENGTH)) + 1

print("MODE:", MODE)
print("TOP_N_CANDIDATES:", TOP_N_CANDIDATES)
print("WINDOW_SECONDS:", WINDOW_SECONDS)
print("MAX_WINDOWS:", MAX_WINDOWS)
print("WINDOW_SELECTION_MODE:", WINDOW_SELECTION_MODE)
print("MIN_WINDOW_VOTES:", MIN_WINDOW_VOTES)
print("REQUIRE_ANCHOR_PREDICTED:", REQUIRE_ANCHOR_PREDICTED)
```

```
print("MIN_STAGE2_SCORE:", MIN_STAGE2_SCORE)
print("LOW_CONFIDENCE_THRESHOLD:", LOW_CONFIDENCE_THRESHOLD)
print("SUPPRESS_STAGE2_IF_LOW_CONFIDENCE:", SUPPRESS_STAGE2_IF_LOW_CONFIDENCE)
print("SR:", SR)
print("N_MELS:", N_MELS)
print("MAX_FRAMES:", MAX_FRAMES)
```

```
MODE: external_file
TOP_N_CANDIDATES: 10
WINDOW_SECONDS: 15
MAX_WINDOWS: 4
WINDOW_SELECTION_MODE: evenly_spaced
MIN_WINDOW_VOTES: 2
REQUIRE_ANCHOR_PREDICTED: True
MIN_STAGE2_SCORE: 0.01
LOW_CONFIDENCE_THRESHOLD: 0.5
SUPPRESS_STAGE2_IF_LOW_CONFIDENCE: False
SR: 22050
N_MELS: 64
MAX_FRAMES: 324
```

In [4]:

```
# ============================================================
# 3. LOAD FROZEN ARTIFACTS
# ============================================================

PROCESSED_DIR = "../data/processed"
MODELS_DIR = "../models"
RAW_METADATA_DIR = "../data/raw/metadata"

structured_model = joblib.load(
    f"{MODELS_DIR}/final_structured_multilabel_candidate150_best_model.joblib"
)

structured_scaler = joblib.load(
    f"{MODELS_DIR}/final_structured_multilabel_candidate150_scaler.joblib"
)

audio_model = tf.keras.models.load_model(
    f"{MODELS_DIR}/audio_multilabel_candidate150_expanded_final.keras"
)

candidate_label_cols = np.load(
    f"{PROCESSED_DIR}/hybrid_multilabel_candidate150_expanded_label_columns.npy",
    allow_pickle=True
)

with open(f"{PROCESSED_DIR}/final_project_frozen_config.json", "r") as f:
    final_project_config = json.load(f)

STAGE1_STRUCTURED_WEIGHT = float(final_project_config["stage1_structured_weight"])
STAGE1_AUDIO_WEIGHT = float(final_project_config["stage1_audio_weight"])
STAGE1_THRESHOLD = float(final_project_config["stage1_threshold"])

STAGE2_FINAL_STRATEGY = final_project_config["stage2_final_strategy"]
```

```
STAGE2_ANCHOR_TRIGGER_THRESHOLD =
float(final_project_config["stage2_anchor_trigger_threshold"])
STAGE2_TOP_K = int(final_project_config["stage2_top_k"])

rare_tail_router_df = pd.read_csv(
    f"{PROCESSED_DIR}/full161_rare_tail_routing_table.csv"
)

genre_inventory_df = pd.read_csv(
    f"{PROCESSED_DIR}/full_genre_inventory.csv"
)

full_master_df = pd.read_csv(
    f"{PROCESSED_DIR}/multilabel_full_master_table.csv"
)

features_reference = pd.read_csv(
    f"{RAW_METADATA_DIR}/features.csv",
    header=[0, 1, 2],
    index_col=0
)

print("Structured model loaded.")
print("Structured scaler loaded.")
print("Audio model loaded.")
print("Candidate labels:", len(candidate_label_cols))
print("Final frozen config loaded.")
print("Rare-tail router shape:", rare_tail_router_df.shape)
print("Genre inventory shape:", genre_inventory_df.shape)
print("Full master shape:", full_master_df.shape)
print("Reference features shape:", features_reference.shape)
```

```
Structured model loaded.
Structured scaler loaded.
Audio model loaded.
Candidate labels: 150
Final frozen config loaded.
Rare-tail router shape: (13, 26)
Genre inventory shape: (163, 11)
Full master shape: (81574, 170)
Reference features shape: (106574, 518)
```

In [5]:

```
# ============================================================
# 4. PREPARE LOOKUPS
# ============================================================

genre_inventory_df["genre_id"] = genre_inventory_df["genre_id"].astype(int)

genre_name_map = dict(
    zip(
        genre_inventory_df["genre_id"],
        genre_inventory_df["genre_name"]
    )
)
```

```
candidate_label_ids = [
    int(col.replace("genre_", ""))
    for col in candidate_label_cols
]

candidate_id_to_index = {
    int(col.replace("genre_", "")): i
    for i, col in enumerate(candidate_label_cols)
}

fallback_router_df = rare_tail_router_df[
    rare_tail_router_df["fallback_mode"] == "Hierarchy-triggered fallback"
].copy().reset_index(drop=True)

fallback_router_df["rare_tail_genre_id"] =
fallback_router_df["rare_tail_genre_id"].astype(int)
fallback_router_df["anchor_candidate_id"] =
fallback_router_df["anchor_candidate_id"].astype(int)

inventory_only_df = rare_tail_router_df[
    rare_tail_router_df["fallback_mode"] == "Inventory only"
].copy().reset_index(drop=True)

# Flatten the multi-index feature columns from FMA's features.csv
features_reference.columns = [
    "_".join([str(level) for level in col]).strip()
    for col in features_reference.columns.to_flat_index()
]

reference_feature_df = features_reference.copy()
reference_feature_df.index = reference_feature_df.index.astype(int)
reference_feature_df = reference_feature_df.select_dtypes(include=["number"])
reference_feature_df = reference_feature_df.replace([np.inf, -np.inf], np.nan)

reference_feature_means = reference_feature_df.mean(axis=0)
reference_feature_columns = list(reference_feature_df.columns)

full_master_indexed = full_master_df.set_index("track_id", drop=False)

fallback_rare_ids = fallback_router_df["rare_tail_genre_id"].astype(int).tolist()
fallback_rare_cols = [f"genre_{gid}" for gid in fallback_rare_ids]

print("Fallback rare-tail labels:", len(fallback_rare_ids))
print("Inventory-only rare-tail labels:", inventory_only_df.shape[0])
print("Structured reference feature columns:", len(reference_feature_columns))

display(fallback_router_df.head())
display(inventory_only_df.head())
```

```
Fallback rare-tail labels: 10
Inventory-only rare-tail labels: 3
Structured reference feature columns: 518
```

	rare_tail_genre_id	rare_tail_genre_name	parent_id	parent_name	root_genre_id	root_genre_
0	176	Pacific	2	International	2	Interna
1	1060	Tango	46	Latin America	2	Interna
2	465	Musical Theater	20	Spoken	20	Sp
3	189	Talk Radio	65	Radio	20	Sp
4	1032	Turkish	102	Middle East	2	Interna

5 rows × 26 columns

	rare_tail_genre_id	rare_tail_genre_name	parent_id	parent_name	root_genre_id	root_genre_
0	174	South Indian Traditional	86	Indian	2	Interna
1	175	Bollywood	86	Indian	2	Interna
2	178	Be-Bop	4	Jazz	4	

3 rows × 26 columns

In [6]:

```
# ============================================================
# 5. RESOLVE INPUT AUDIO
# ============================================================

if MODE == "existing_fma_track":
    if TRACK_ID_TO_TEST not in full_master_indexed.index:
        raise ValueError(f"track_id {TRACK_ID_TO_TEST} not found in full_master
table.")

    input_row = full_master_indexed.loc[TRACK_ID_TO_TEST]
    AUDIO_PATH = input_row["audio_path"]
    ACTIVE_TRACK_ID = int(input_row["track_id"])
    INPUT_SOURCE = "existing_fma_track"

elif MODE == "external_file":
    AUDIO_PATH = EXTERNAL_AUDIO_PATH
    ACTIVE_TRACK_ID = None
    INPUT_SOURCE = "external_file"
```

```
else:
    raise ValueError("MODE must be either 'existing_fma_track' or
'external_file'.")

print("Input source:", INPUT_SOURCE)
print("Audio path:", AUDIO_PATH)
print("Track ID:", ACTIVE_TRACK_ID)
```

```
Input source: external_file
Audio path: C:\Users\jdevo\Downloads\Bob Marley - Is This Love (Official Music
Video).mp3
Track ID: None
```

In [7]:

```
# ============================================================
# 6. HELPER FUNCTIONS
# ============================================================

def load_audio_robust(file_path, sr=SR):
    """
    Load full audio file.

    First tries librosa directly.
    If that fails for MP4/M4A-style container formats,
    it tries to convert the file to a temporary WAV using ffmpeg.
    """
    try:
        y, sr_loaded = librosa.load(file_path, sr=sr, mono=True)

        if y is None or len(y) == 0:
            raise ValueError(f"Loaded audio is empty: {file_path}")

        return y, sr_loaded

    except Exception as first_error:
        ext = os.path.splitext(file_path)[1].lower()
        fallback_exts = {".mp4", ".m4a", ".aac", ".mov", ".3gp", ".webm"}

        if ext not in fallback_exts:
            raise RuntimeError(
                f"Could not load audio file: {file_path}\n"
                f"Original error: {first_error}"
            )

        try:
            with tempfile.NamedTemporaryFile(suffix=".wav", delete=False) as tmp:
                tmp_wav = tmp.name

            cmd = [
                "ffmpeg",
                "-y",
                "-i",
                file_path,
                "-ac",
```

```
                "1",
                "-ar",
                str(sr),
                tmp_wav
            ]

            result = subprocess.run(
                cmd,
                stdout=subprocess.PIPE,
                stderr=subprocess.PIPE,
                text=True
            )

            if result.returncode != 0:
                raise RuntimeError(result.stderr)

            y, sr_loaded = librosa.load(tmp_wav, sr=sr, mono=True)

            try:
                os.remove(tmp_wav)
            except Exception:
                pass

            if y is None or len(y) == 0:
                raise ValueError(f"Converted audio is empty: {file_path}")

            return y, sr_loaded

        except Exception as second_error:
            raise RuntimeError(
                f"Could not decode audio file: {file_path}\n\n"
                f"Direct librosa load error:\n{first_error}\n\n"
                f"FFmpeg fallback error:\n{second_error}\n\n"
                f"Recommended fix: convert the file to WAV first, then rerun the
pipeline."
            )


def get_window_start_samples(
    y,
    sr=SR,
    window_seconds=WINDOW_SECONDS,
    max_windows=MAX_WINDOWS,
    mode=WINDOW_SELECTION_MODE
):
    """
    Select start positions for multiple windows across the audio.
    """
    total_len = len(y)
    window_len = int(window_seconds * sr)

    if total_len <= window_len or max_windows <= 1:
        return [0]

    max_start = total_len - window_len

    if mode == "first_n":
```

```
        starts = []
        current = 0

        while current <= max_start and len(starts) < max_windows:
            starts.append(int(current))
            current += window_len

        if len(starts) == 0:
            starts = [0]

        return starts

    if mode == "evenly_spaced":
        n_windows = min(max_windows, max(2, math.ceil(total_len / window_len)))
        starts = np.linspace(0, max_start, num=n_windows)
        starts = [int(x) for x in starts]
        starts = sorted(list(dict.fromkeys(starts)))[:max_windows]
        return starts

    raise ValueError("WINDOW_SELECTION_MODE must be 'evenly_spaced' or 'first_n'.")


def extract_window(y, start_sample, sr=SR, window_seconds=WINDOW_SECONDS):
    """
    Extract a fixed-length audio window.
    Pads with silence if the audio is shorter than the window length.
    """
    window_len = int(window_seconds * sr)
    segment = y[start_sample:start_sample + window_len]

    if len(segment) < window_len:
        segment = np.pad(segment, (0, window_len - len(segment)), mode="constant")

    return segment.astype(np.float32)


def build_mel_input(
    y_segment,
    sr=SR,
    n_mels=N_MELS,
    n_fft=N_FFT,
    hop_length=HOP_LENGTH,
    max_frames=MAX_FRAMES
):
    """
    Convert an audio window into a Mel spectrogram image for the CNN.
    """
    mel = librosa.feature.melspectrogram(
        y=y_segment,
        sr=sr,
        n_fft=n_fft,
        hop_length=hop_length,
        n_mels=n_mels
    )

    mel_db = librosa.power_to_db(mel, ref=np.max)
    mel_db = np.clip((mel_db + 80.0) / 80.0, 0.0, 1.0)
```

```
    if mel_db.shape[1] < max_frames:
        pad_width = max_frames - mel_db.shape[1]
        mel_db = np.pad(mel_db, ((0, 0), (0, pad_width)), mode="constant")
    else:
        mel_db = mel_db[:, :max_frames]

    return mel_db.astype(np.float32)[None, :, :, None]


def safe_stat_vector(arr_2d, stat_name):
    """
    Compute statistical summaries safely.
    Converts to float64 during computation to avoid casting issues,
    then returns float32.
    """
    arr_2d = np.asarray(arr_2d, dtype=np.float64)

    if arr_2d.ndim == 1:
        arr_2d = arr_2d.reshape(1, -1)

    if stat_name == "mean":
        out = np.mean(arr_2d, axis=1)

    elif stat_name == "std":
        out = np.std(arr_2d, axis=1)

    elif stat_name == "median":
        out = np.median(arr_2d, axis=1)

    elif stat_name == "min":
        out = np.min(arr_2d, axis=1)

    elif stat_name == "max":
        out = np.max(arr_2d, axis=1)

    elif stat_name == "skew":
        out = skew(arr_2d, axis=1, bias=False, nan_policy="omit")

    elif stat_name == "kurtosis":
        out = kurtosis(arr_2d, axis=1, bias=False, nan_policy="omit")

    else:
        raise ValueError(f"Unknown stat: {stat_name}")

    out = np.asarray(out, dtype=np.float64)
    out[~np.isfinite(out)] = 0.0

    return out.astype(np.float32)


def build_feature_matrices(y_segment, sr=SR):
    """
    Extract the same broad family of audio descriptors used by FMA features.
    """
    y_segment = np.asarray(y_segment, dtype=np.float64)
```

```
    try:
        y_harmonic = librosa.effects.harmonic(y_segment)
    except Exception:
        y_harmonic = y_segment

    mats = {}

    mats["chroma_stft"] = librosa.feature.chroma_stft(y=y_segment, sr=sr)
    mats["chroma_cqt"] = librosa.feature.chroma_cqt(y=y_segment, sr=sr)
    mats["chroma_cens"] = librosa.feature.chroma_cens(y=y_segment, sr=sr)
    mats["tonnetz"] = librosa.feature.tonnetz(y=y_harmonic, sr=sr)
    mats["mfcc"] = librosa.feature.mfcc(y=y_segment, sr=sr, n_mfcc=20)
    mats["rms"] = librosa.feature.rms(y=y_segment)
    mats["spectral_centroid"] = librosa.feature.spectral_centroid(y=y_segment,
sr=sr)
    mats["spectral_bandwidth"] = librosa.feature.spectral_bandwidth(y=y_segment,
sr=sr)
    mats["spectral_contrast"] = librosa.feature.spectral_contrast(y=y_segment,
sr=sr)
    mats["spectral_rolloff"] = librosa.feature.spectral_rolloff(y=y_segment, sr=sr)
    mats["zcr"] = librosa.feature.zero_crossing_rate(y_segment)

    return mats


def build_structured_feature_vector(
    y_segment,
    reference_columns,
    reference_means,
    sr=SR
):
    """
    Build one structured feature vector from an audio window.
    This approximates the structure of FMA's precomputed feature file.
    """
    feature_mats = build_feature_matrices(y_segment, sr=sr)
    row_dict = {}

    for col in reference_columns:
        parts = col.split("_")

        try:
            component_idx = int(parts[-1]) - 1
            stat_name = parts[-2]
            feature_name = "_".join(parts[:-2])
        except Exception:
            row_dict[col] = np.nan
            continue

        if feature_name in feature_mats:
            mat = feature_mats[feature_name]
            stat_vec = safe_stat_vector(mat, stat_name)

            if 0 <= component_idx < len(stat_vec):
                row_dict[col] = float(stat_vec[component_idx])
            else:
                row_dict[col] = np.nan
```

```
        else:
            row_dict[col] = np.nan

    X_one = pd.DataFrame([row_dict], columns=reference_columns)
    X_one = X_one.replace([np.inf, -np.inf], np.nan)

    for col in reference_columns:
        if pd.isna(X_one.loc[0, col]):
            X_one.loc[0, col] = float(reference_means[col])

    return X_one.astype(np.float32)


def scores_to_pseudoprobs(score_matrix):
    """
    Convert structured model scores into pseudo-probabilities.
    """
    clipped = np.clip(score_matrix, -20, 20)
    return 1.0 / (1.0 + np.exp(-clipped))


def get_structured_scores(model, X_scaled):
    """
    Get output scores from the structured model.
    """
    if hasattr(model, "decision_function"):
        scores = model.decision_function(X_scaled)

    elif hasattr(model, "predict_proba"):
        scores = model.predict_proba(X_scaled)

    else:
        raise ValueError("Structured model supports neither decision_function nor
predict_proba.")

    return np.asarray(scores)


def fuse_probabilities(structured_probs, audio_probs, w_structured, w_audio):
    """
    Weighted average fusion of structured and audio probabilities.
    """
    return (w_structured * structured_probs) + (w_audio * audio_probs)


def decode_stage1(prob_matrix, threshold):
    """
    Convert probabilities into binary multi-label predictions.
    """
    return (prob_matrix >= threshold).astype(np.uint8)


def build_stage2_scores_baseline_prior(
    stage1_probs,
    stage1_pred,
    router_df,
    candidate_index_map,
```

```
    trigger_threshold,
    min_stage2_score,
    require_anchor_predicted=True
):
    """
    Safer Stage-2 rare-tail fallback scoring.

    A rare-tail suggestion is eligible only if:
    - its anchor probability passes the trigger threshold
    - the rare-tail fallback score passes the minimum score
    - optionally, the anchor was predicted by Stage 1
    """
    rows = []

    for _, row in router_df.iterrows():
        anchor_id = int(row["anchor_candidate_id"])
        anchor_idx = candidate_index_map[anchor_id]

        anchor_prob = float(stage1_probs[0, anchor_idx])
        anchor_predicted = int(stage1_pred[0, anchor_idx])

        p_anchor = 0.0 if pd.isna(row["p_rare_given_anchor"]) else
float(row["p_rare_given_anchor"])
        p_root = 0.0 if pd.isna(row["p_rare_given_root"]) else
float(row["p_rare_given_root"])

        prior_strength = max(p_anchor, p_root)
        stage2_score = anchor_prob * prior_strength

        passes_anchor_threshold = anchor_prob >= trigger_threshold
        passes_min_score = stage2_score >= min_stage2_score
        passes_anchor_predicted = (anchor_predicted == 1) if
require_anchor_predicted else True

        eligible = (
            passes_anchor_threshold and
            passes_min_score and
            passes_anchor_predicted
        )

        rows.append({
            "rare_tail_genre_id": int(row["rare_tail_genre_id"]),
            "rare_tail_genre_name": row["rare_tail_genre_name"],
            "anchor_candidate_id": anchor_id,
            "anchor_candidate_name": row["anchor_candidate_name"],
            "anchor_prob": anchor_prob,
            "anchor_predicted": anchor_predicted,
            "prior_strength": prior_strength,
            "stage2_score": stage2_score,
            "passes_anchor_threshold": passes_anchor_threshold,
            "passes_min_score": passes_min_score,
            "passes_anchor_predicted": passes_anchor_predicted,
            "eligible_stage2": eligible
        })

    all_scores_df = pd.DataFrame(rows).sort_values(
        ["eligible_stage2", "stage2_score", "anchor_prob"],
```

```
        ascending=[False, False, False]
    ).reset_index(drop=True)

    suggestions_df = all_scores_df[
        all_scores_df["eligible_stage2"] == True
    ].copy().reset_index(drop=True)

    return all_scores_df, suggestions_df


def get_ground_truth_for_fma_track(track_id):
    """
    Return ground truth labels for an FMA track.
    Only works when testing an existing FMA track.
    """
    row = full_master_indexed.loc[track_id]

    true_candidate_ids = []

    for col in candidate_label_cols:
        if int(row[col]) == 1:
            true_candidate_ids.append(int(col.replace("genre_", "")))

    true_candidate_names = [
        genre_name_map.get(gid, str(gid))
        for gid in true_candidate_ids
    ]

    true_rare_ids = []

    for col in fallback_rare_cols:
        if col in row.index and int(row[col]) == 1:
            true_rare_ids.append(int(col.replace("genre_", "")))

    true_rare_names = [
        genre_name_map.get(gid, str(gid))
        for gid in true_rare_ids
    ]

    return {
        "track_id": int(track_id),
        "true_candidate_ids": true_candidate_ids,
        "true_candidate_names": true_candidate_names,
        "true_rare_tail_ids": true_rare_ids,
        "true_rare_tail_names": true_rare_names
    }
```

In [8]:

```
# ============================================================
# 7. LOAD FULL AUDIO AND BUILD WINDOWS
# ============================================================

y_full, sr_loaded = load_audio_robust(AUDIO_PATH, sr=SR)
```

```
window_starts = get_window_start_samples(
    y_full,
    sr=sr_loaded,
    window_seconds=WINDOW_SECONDS,
    max_windows=MAX_WINDOWS,
    mode=WINDOW_SELECTION_MODE
)

window_info_rows = []
window_segments = []

for i, start_sample in enumerate(window_starts):
    segment = extract_window(
        y_full,
        start_sample,
        sr=sr_loaded,
        window_seconds=WINDOW_SECONDS
    )

    start_sec = start_sample / sr_loaded
    end_sec = start_sec + WINDOW_SECONDS

    window_segments.append(segment)

    window_info_rows.append({
        "window_index": i,
        "start_sample": start_sample,
        "start_second": round(start_sec, 2),
        "end_second": round(end_sec, 2),
        "window_length_samples": len(segment)
    })

window_info_df = pd.DataFrame(window_info_rows)

print("Loaded full audio length (samples):", len(y_full))
print("Loaded full audio length (seconds):", round(len(y_full) / sr_loaded, 2))
print("Number of windows:", len(window_segments))

display(window_info_df)
```

```
Loaded full audio length (samples): 5154816
Loaded full audio length (seconds): 233.78
Number of windows: 4
```

	window_index	start_sample	start_second	end_second	window_length_samples
0	0	0	0.00	15.00	330750
1	1	1608022	72.93	87.93	330750
2	2	3216044	145.85	160.85	330750
3	3	4824066	218.78	233.78	330750

In [9]:

```
# ============================================================
# 8. RUN MULTI-WINDOW INFERENCE
# ============================================================

window_result_rows = []

structured_prob_list = []
audio_prob_list = []
hybrid_prob_list = []

for i, segment in enumerate(window_segments):
    print(f"Processing window {i + 1} of {len(window_segments)}...")

    X_audio_input = build_mel_input(
        segment,
        sr=sr_loaded,
        n_mels=N_MELS,
        n_fft=N_FFT,
        hop_length=HOP_LENGTH,
        max_frames=MAX_FRAMES
    )

    X_structured_one = build_structured_feature_vector(
        segment,
        reference_feature_columns,
        reference_feature_means,
        sr=sr_loaded
    )

    X_structured_scaled =
structured_scaler.transform(X_structured_one).astype(np.float32)

    structured_scores = get_structured_scores(
        structured_model,
        X_structured_scaled
    )

    structured_probs = scores_to_pseudoprobs(structured_scores)

    audio_probs = audio_model.predict(
        X_audio_input,
        verbose=0
    )

    hybrid_probs = fuse_probabilities(
        structured_probs,
        audio_probs,
        STAGE1_STRUCTURED_WEIGHT,
        STAGE1_AUDIO_WEIGHT
    )

    structured_prob_list.append(structured_probs[0])
    audio_prob_list.append(audio_probs[0])
    hybrid_prob_list.append(hybrid_probs[0])

    top_idx = int(np.argmax(hybrid_probs[0]))
```

```
    top_gid = candidate_label_ids[top_idx]
    top_name = genre_name_map.get(top_gid, str(top_gid))

    window_result_rows.append({
        "window_index": i,
        "top_genre_id": top_gid,
        "top_genre_name": top_name,
        "top_hybrid_probability": float(hybrid_probs[0, top_idx])
    })

window_results_df = pd.DataFrame(window_result_rows)

structured_probs_avg = np.mean(
    np.vstack(structured_prob_list),
    axis=0,
    keepdims=True
)

audio_probs_avg = np.mean(
    np.vstack(audio_prob_list),
    axis=0,
    keepdims=True
)

stage1_probs = np.mean(
    np.vstack(hybrid_prob_list),
    axis=0,
    keepdims=True
)

stage1_pred_raw = decode_stage1(stage1_probs, STAGE1_THRESHOLD)

print("Structured averaged probability shape:", structured_probs_avg.shape)
print("Audio averaged probability shape:", audio_probs_avg.shape)
print("Stage-1 averaged fused probability shape:", stage1_probs.shape)
print("Raw Stage-1 hard prediction shape:", stage1_pred_raw.shape)
print("Raw number of Stage-1 predicted labels:", int(stage1_pred_raw.sum()))

print("Per-window top label summary:")
display(window_results_df)
```

```
Processing window 1 of 4...
Processing window 2 of 4...
Processing window 3 of 4...
Processing window 4 of 4...
Structured averaged probability shape: (1, 150)
Audio averaged probability shape: (1, 150)
Stage-1 averaged fused probability shape: (1, 150)
Raw Stage-1 hard prediction shape: (1, 150)
Raw number of Stage-1 predicted labels: 3
Per-window top label summary:
```

	window_index	top_genre_id	top_genre_name	top_hybrid_probability
0	0	12	Rock	0.413449
1	1	12	Rock	0.536982
2	2	12	Rock	0.516510
3	3	12	Rock	0.494895

In [10]:

```
# ============================================================
# 9. BUILD STAGE-1 RESULTS TABLES WITH WINDOW AGGREGATION
# ============================================================
# This section combines predictions from all windows.
#
# It checks:
# 1. The average hybrid confidence score across windows
# 2. How many windows predicted each genre
#
# This makes the final output more stable than using only one
# 15-second section of the song.

window_hybrid_probs = np.vstack(hybrid_prob_list)
window_structured_probs = np.vstack(structured_prob_list)
window_audio_probs = np.vstack(audio_prob_list)

window_pred_matrix = (window_hybrid_probs >= STAGE1_THRESHOLD).astype(int)

# If the song has fewer than MIN_WINDOW_VOTES windows, adjust automatically.
# Example: if only 1 window exists, only 1 vote is required.
required_window_votes = min(MIN_WINDOW_VOTES, len(window_segments))

aggregation_rows = []

for j, col in enumerate(candidate_label_cols):
    genre_id = int(col.replace("genre_", ""))
    genre_name = genre_name_map.get(genre_id, str(genre_id))

    structured_scores_for_genre = window_structured_probs[:, j]
    audio_scores_for_genre = window_audio_probs[:, j]
    hybrid_scores_for_genre = window_hybrid_probs[:, j]
    predictions_for_genre = window_pred_matrix[:, j]

    window_vote_count = int(predictions_for_genre.sum())
    window_vote_rate = window_vote_count / len(window_segments)

    mean_structured_score = float(np.mean(structured_scores_for_genre))
    mean_audio_score = float(np.mean(audio_scores_for_genre))
    mean_hybrid_score = float(np.mean(hybrid_scores_for_genre))

    max_hybrid_score = float(np.max(hybrid_scores_for_genre))
    min_hybrid_score = float(np.min(hybrid_scores_for_genre))

    final_predicted_by_mean = int(mean_hybrid_score >= STAGE1_THRESHOLD)
```

```
    final_predicted_by_vote = int(window_vote_count >= required_window_votes)

    # Final rule:
    # Genre must have enough average confidence AND appear in enough windows.
    final_pipeline_prediction = int(
        (mean_hybrid_score >= STAGE1_THRESHOLD) and
        (window_vote_count >= required_window_votes)
    )

    aggregation_rows.append({
        "genre_id": genre_id,
        "genre_name": genre_name,
        "structured_probability_avg": mean_structured_score,
        "audio_probability_avg": mean_audio_score,
        "hybrid_probability_avg": mean_hybrid_score,
        "max_hybrid_probability": max_hybrid_score,
        "min_hybrid_probability": min_hybrid_score,
        "window_vote_count": window_vote_count,
        "window_vote_rate": window_vote_rate,
        "required_window_votes": required_window_votes,
        "final_predicted_by_mean": final_predicted_by_mean,
        "final_predicted_by_vote": final_predicted_by_vote,
        "predicted_stage1": final_pipeline_prediction
    })

candidate_results_df = pd.DataFrame(aggregation_rows).sort_values(
    [
        "predicted_stage1",
        "window_vote_count",
        "hybrid_probability_avg"
    ],
    ascending=[False, False, False]
).reset_index(drop=True)

predicted_candidate_df = candidate_results_df[
    candidate_results_df["predicted_stage1"] == 1
].copy().reset_index(drop=True)

top_candidate_df = candidate_results_df.head(TOP_N_CANDIDATES).copy()

# Convert final predictions back into the matrix format needed by Stage 2
stage1_pred_final = np.zeros(
    (1, len(candidate_label_cols)),
    dtype=np.uint8
)

for _, row in predicted_candidate_df.iterrows():
    genre_id = int(row["genre_id"])
    genre_index = candidate_id_to_index[genre_id]
    stage1_pred_final[0, genre_index] = 1

top_stage1_probability = float(candidate_results_df.iloc[0]
["hybrid_probability_avg"])
low_confidence_flag = top_stage1_probability < LOW_CONFIDENCE_THRESHOLD

print("Required window votes:", required_window_votes)
```

```
print("Predicted Stage-1 candidate labels using mean score + window votes:")
display(predicted_candidate_df)

print(f"Top {TOP_N_CANDIDATES} candidate labels by averaged hybrid probability:")
display(top_candidate_df)

print("Top Stage-1 probability:", round(top_stage1_probability, 6))
print("Low confidence flag:", low_confidence_flag)
```

```
Required window votes: 2
Predicted Stage-1 candidate labels using mean score + window votes:
```

	genre_id	genre_name	structured_probability_avg	audio_probability_avg	hybrid_probability_
0	12	Rock	0.634197	0.474488	0.490
1	38	Experimental	0.236415	0.277669	0.273
2	10	Pop	0.258194	0.205493	0.210

```
Top 10 candidate labels by averaged hybrid probability:
```

	genre_id	genre_name	structured_probability_avg	audio_probability_avg	hybrid_probability_
0	12	Rock	6.341970e-01	0.474488	0.490
1	38	Experimental	2.364150e-01	0.277669	0.273
2	10	Pop	2.581943e-01	0.205493	0.210
3	17	Folk	5.194664e-01	0.149020	0.186
4	15	Electronic	2.559845e-03	0.193407	0.174
5	21	Hip-Hop	1.381421e-08	0.140614	0.126
6	250	Improv	5.050416e-01	0.061124	0.105
7	25	Punk	2.061154e-09	0.171479	0.154
8	76	Experimental Pop	5.095955e-01	0.089280	0.131
9	2	International	2.441295e-01	0.113924	0.126

```
Top Stage-1 probability: 0.490459
Low confidence flag: True
```

In [11]:

```
# ============================================================
# 10. RUN SAFER STAGE-2
# ============================================================

stage2_scores_df, stage2_suggestions_df = build_stage2_scores_baseline_prior(
    stage1_probs=stage1_probs,
    stage1_pred=stage1_pred_final,
    router_df=fallback_router_df,
```

```
    candidate_index_map=candidate_id_to_index,
    trigger_threshold=STAGE2_ANCHOR_TRIGGER_THRESHOLD,
    min_stage2_score=MIN_STAGE2_SCORE,
    require_anchor_predicted=REQUIRE_ANCHOR_PREDICTED
)

if SUPPRESS_STAGE2_IF_LOW_CONFIDENCE and low_confidence_flag:
    stage2_suggestions_df = stage2_suggestions_df.iloc[0:0].copy()

stage2_suggestions_df =
stage2_suggestions_df.head(STAGE2_TOP_K).copy().reset_index(drop=True)

print("All Stage-2 rare-tail scores:")
display(stage2_scores_df)

print("Final safer Stage-2 rare-tail suggestions:")
display(stage2_suggestions_df)
```

```
All Stage-2 rare-tail scores:
```

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_id	anchor_candidate_name	ancho
0	176	Pacific	2	International	0.
1	1032	Turkish	102	Middle East	0.
2	1060	Tango	46	Latin America	0.
3	374	Banter	20	Spoken	0.
4	189	Talk Radio	65	Radio	0.
5	465	Musical Theater	20	Spoken	0.
6	377	Deep Funk	19	Funk	0.
7	808	Salsa	46	Latin America	0.
8	173	N. Indian Traditional	86	Indian	0.
9	493	Western Swing	651	Country & Western	0.

```
Final safer Stage-2 rare-tail suggestions:
```

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_id	anchor_candidate_name	anchor

In [12]:

```
# ============================================================
# 11. INVENTORY-ONLY LABELS
# ============================================================

inventory_only_labels_df = inventory_only_df[
    [
        "rare_tail_genre_id",
        "rare_tail_genre_name",
```

```
        "anchor_candidate_name",
        "root_candidate_name",
        "fallback_mode"
    ]
].copy()

print("Inventory-only rare-tail labels:")
display(inventory_only_labels_df)
```

Inventory-only rare-tail labels:

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_name	root_candidate_name	fallb
0	174	South Indian Traditional	Indian	International	Inve
1	175	Bollywood	Indian	International	Inve
2	178	Be-Bop	Jazz	Jazz	Inve

In [13]:

```
# ============================================================
# 12. GROUND TRUTH CHECK FOR FMA TRACKS
# ============================================================

ground_truth = None

if INPUT_SOURCE == "existing_fma_track" and ACTIVE_TRACK_ID is not None:
    ground_truth = get_ground_truth_for_fma_track(ACTIVE_TRACK_ID)

    print("Ground truth for selected FMA track:")
    print(json.dumps(ground_truth, indent=2))

else:
    print("No ground truth shown because MODE is external_file.")
```

```
No ground truth shown because MODE is external_file.
```

In [14]:

```
# ============================================================
# 13. BUILD FINAL PIPELINE SUMMARY
# ============================================================

stage1_candidate_ids = predicted_candidate_df["genre_id"].astype(int).tolist()
stage1_candidate_names = predicted_candidate_df["genre_name"].tolist()

if len(stage2_suggestions_df) > 0:
    stage2_rare_tail_ids =
stage2_suggestions_df["rare_tail_genre_id"].astype(int).tolist()
    stage2_rare_tail_names = stage2_suggestions_df["rare_tail_genre_name"].tolist()
    stage2_rare_tail_scores =
stage2_suggestions_df["stage2_score"].round(6).tolist()
```

```
else:
    stage2_rare_tail_ids = []
    stage2_rare_tail_names = []
    stage2_rare_tail_scores = []

final_summary = {
    "input_source": INPUT_SOURCE,
    "audio_path": AUDIO_PATH,
    "track_id": ACTIVE_TRACK_ID,

    "multi_window_config": {
        "window_seconds": WINDOW_SECONDS,
        "max_windows": MAX_WINDOWS,
        "window_selection_mode": WINDOW_SELECTION_MODE,
        "windows_used": len(window_segments),
        "min_window_votes": MIN_WINDOW_VOTES,
        "required_window_votes": required_window_votes,
        "final_prediction_rule": (
            "mean_hybrid_score >= threshold "
            "AND window_vote_count >= required_window_votes"
        )
    },

    "stage1_config": {
        "model_name": final_project_config["stage1_primary_system_name"],
        "structured_weight": STAGE1_STRUCTURED_WEIGHT,
        "audio_weight": STAGE1_AUDIO_WEIGHT,
        "threshold": STAGE1_THRESHOLD
    },

    "stage2_config": {
        "router_name": final_project_config["stage2_router_name"],
        "strategy": STAGE2_FINAL_STRATEGY,
        "anchor_trigger_threshold": STAGE2_ANCHOR_TRIGGER_THRESHOLD,
        "min_stage2_score": MIN_STAGE2_SCORE,
        "top_k": STAGE2_TOP_K,
        "require_anchor_predicted": REQUIRE_ANCHOR_PREDICTED,
        "suppress_if_low_confidence": SUPPRESS_STAGE2_IF_LOW_CONFIDENCE
    },

    "top_stage1_probability": top_stage1_probability,
    "low_confidence_flag": bool(low_confidence_flag),

    "stage1_candidate_label_count": len(stage1_candidate_ids),
    "stage1_candidate_label_ids": stage1_candidate_ids,
    "stage1_candidate_label_names": stage1_candidate_names,

    "stage2_rare_tail_suggestion_count": len(stage2_rare_tail_ids),
    "stage2_rare_tail_suggestion_ids": stage2_rare_tail_ids,
    "stage2_rare_tail_suggestion_names": stage2_rare_tail_names,
    "stage2_rare_tail_suggestion_scores": stage2_rare_tail_scores,

    "inventory_only_rare_tail_labels":
inventory_only_labels_df["rare_tail_genre_name"].tolist()
}

if ground_truth is not None:
```

```
    final_summary["ground_truth"] = ground_truth

print("Final pipeline summary:")
print(json.dumps(final_summary, indent=2))
```

```
Final pipeline summary:
{
  "input_source": "external_file",
  "audio_path": "C:\\Users\\jdevo\\Downloads\\Bob Marley - Is This Love (Official
Music Video).mp3",
  "track_id": null,
  "multi_window_config": {
    "window_seconds": 15,
    "max_windows": 4,
    "window_selection_mode": "evenly_spaced",
    "windows_used": 4,
    "min_window_votes": 2,
    "required_window_votes": 2,
    "final_prediction_rule": "mean_hybrid_score >= threshold AND window_vote_count >=
required_window_votes"
  },
  "stage1_config": {
    "model_name": "Expanded Hybrid Global Threshold",
    "structured_weight": 0.1,
    "audio_weight": 0.9,
    "threshold": 0.2
  },
  "stage2_config": {
    "router_name": "Rare-tail fallback router",
    "strategy": "baseline_prior",
    "anchor_trigger_threshold": 0.05,
    "min_stage2_score": 0.01,
    "top_k": 1,
    "require_anchor_predicted": true,
    "suppress_if_low_confidence": false
  },
  "top_stage1_probability": 0.490459106699612,
  "low_confidence_flag": true,
  "stage1_candidate_label_count": 3,
  "stage1_candidate_label_ids": [
    12,
    38,
    10
  ],
  "stage1_candidate_label_names": [
    "Rock",
    "Experimental",
    "Pop"
  ],
  "stage2_rare_tail_suggestion_count": 0,
  "stage2_rare_tail_suggestion_ids": [],
  "stage2_rare_tail_suggestion_names": [],
  "stage2_rare_tail_suggestion_scores": [],
  "inventory_only_rare_tail_labels": [
    "South Indian Traditional",
    "Bollywood",
    "Be-Bop"
  ]
}
```

In [15]:

```
# ============================================================
# 14. SAVE FINAL OUTPUTS
# ============================================================

os.makedirs(PROCESSED_DIR, exist_ok=True)

window_info_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_window_info.csv",
    index=False
)

window_results_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_window_top_results.csv",
    index=False
)

candidate_results_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_candidate_results.csv",
    index=False
)

candidate_results_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_window_aggregation_results.csv",
    index=False
)

predicted_candidate_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_stage1_predictions.csv",
    index=False
)

top_candidate_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_top_candidates.csv",
    index=False
)

stage2_scores_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_stage2_all_scores.csv",
    index=False
)

stage2_suggestions_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_stage2_suggestions.csv",
    index=False
)

inventory_only_labels_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_inventory_only_labels.csv",
    index=False
)

summary_df = pd.DataFrame([{
    "input_source": final_summary["input_source"],
    "audio_path": final_summary["audio_path"],
    "track_id": final_summary["track_id"],
    "windows_used": final_summary["multi_window_config"]["windows_used"],
```

```
    "min_window_votes": final_summary["multi_window_config"]["min_window_votes"],
    "required_window_votes": final_summary["multi_window_config"]
["required_window_votes"],
    "top_stage1_probability": final_summary["top_stage1_probability"],
    "low_confidence_flag": final_summary["low_confidence_flag"],
    "stage1_candidate_label_count": final_summary["stage1_candidate_label_count"],
    "stage2_rare_tail_suggestion_count":
final_summary["stage2_rare_tail_suggestion_count"]
}])

summary_df.to_csv(
    f"{PROCESSED_DIR}/final_pipeline_summary.csv",
    index=False
)

with open(f"{PROCESSED_DIR}/final_pipeline_summary.json", "w") as f:
    json.dump(final_summary, f, indent=2)

print("Saved final patched pipeline inference outputs.")
```

```
Saved final patched pipeline inference outputs.
```

In [ ]:

```
# ============================================================
# 15. INTERPRETATION NOTES
# ============================================================

print("1. This notebook is the patched final deployment-style inference pipeline.")
print("2. Stage 1 now uses multi-window inference instead of only the first 15
seconds.")
print("3. Final Stage-1 predictions now require both average confidence and enough
window votes.")
print("4. Stage 2 now uses safer filtering before surfacing rare-tail
suggestions.")
print("5. Low-confidence cases are flagged so predictions are interpreted more
carefully.")
print("6. This is the version you should use for final real-audio testing before
documentation.")
```

```
1. This notebook is the patched final deployment-style inference pipeline.
2. Stage 1 now uses multi-window inference instead of only the first 15 seconds.
3. Final Stage-1 predictions now require both average confidence and enough window
votes.
4. Stage 2 now uses safer filtering before surfacing rare-tail suggestions.
5. Low-confidence cases are flagged so predictions are interpreted more carefully.
6. This is the version you should use for final real-audio testing before
documentation.
```

```
The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failure.

Click <a href='https://aka.ms/vscodeJupyterKernelCrash'>here</a> for more info.

View Jupyter <a href='command:jupyter.viewOutput'>log</a> for further details.
```